

Linked lists: The role of locking

9.1 Introduction

In Chapter 7, we saw how to build scalable spin locks that provide mutual exclusion efficiently, even when they are heavily used. We might think that it is now a simple matter to construct scalable concurrent data structures: Take a sequential implementation of the class, add a scalable lock field, and ensure that each method call acquires and releases that lock. We call this approach *coarse-grained synchronization*.

Coarse-grained synchronization often works well, but there are important cases where it does not. The problem is that a class that uses a single lock to mediate all its method calls is not always scalable, even if the lock itself is scalable. Coarse-grained synchronization works well when levels of concurrency are low, but if too many threads try to access the object at the same time, then the object becomes a sequential bottleneck, forcing threads to wait in line for access.

This chapter introduces several useful techniques that go beyond coarse-grained locking to allow multiple threads to access a single object at the same time.

- *Fine-grained synchronization*: Instead of using a single lock to synchronize every access to an object, we partition the object into independently synchronized components, allowing method calls that access disjoint components to execute concurrently.
- *Optimistic synchronization*: Many objects, such as trees or lists, consist of multiple components linked together by references. Some methods search for a particular component (e.g., a list or tree node containing a particular key). One way to reduce the cost of fine-grained locking is to search without acquiring any locks at all. If the method finds the sought-after component, it locks that component, and then checks that the component has not changed in the interval between when it was inspected and when it was locked. This technique is worthwhile only if it succeeds more often than not, which is why we call it optimistic.
- *Lazy synchronization*: Sometimes it makes sense to postpone hard work. For example, the task of removing a component from a data structure can be split into two phases: The component is *logically removed* simply by setting a tag bit, and later, the component can be *physically removed* by unlinking it from the rest of the data structure.
- *Nonblocking synchronization*: Sometimes we can eliminate locks entirely, relying on built-in atomic operations such as `compareAndSet()` for synchronization.

```

1 public interface Set<T> {
2     boolean add(T x);
3     boolean remove(T x);
4     boolean contains(T x);
5 }

```

FIGURE 9.1

The `Set<T>` interface: `add()` adds an item to the set (no effect if that item is already present), `remove()` removes it (if present), and `contains()` returns a `Boolean` indicating whether the item is present.

Each of these techniques can be applied (with appropriate customization) to a variety of common data structures. In this chapter, we consider how to use linked lists to implement a *set*, a collection of *items* that contains no duplicate elements.

For our purposes, as shown in Fig. 9.1, a *set* provides the following three methods:

- The `add(x)` method adds x to the set, returning *true* if and only if x was not already in the set.
- The `remove(x)` method removes x from the set, returning *true* if and only if x was in the set.
- The `contains(x)` returns *true* if and only if the set contains x .

For each method, we say that a call is *successful* if it returns *true*, and *unsuccessful* otherwise. In typical applications using sets, there are significantly more `contains()` calls than `add()` or `remove()` calls.

9.2 List-based sets

This chapter presents a range of concurrent set algorithms, all based on the same basic idea. A set is implemented as a linked list of nodes. The `Node<T>` class, shown in Fig. 9.2, has three fields. The `item` field is the actual item of interest. The `key` field is the item's hash code. Nodes are sorted in key order, providing an efficient way to detect when an item is absent. The `next` field is a reference to the next node in the

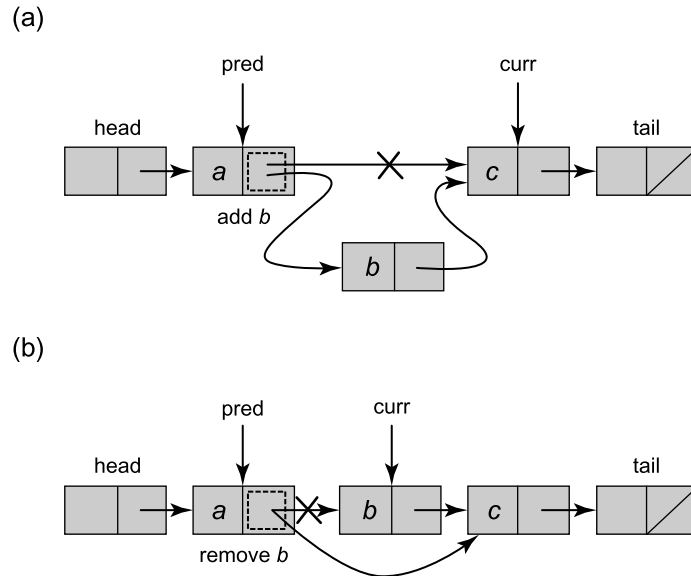
```

1 private class Node {
2     T item;
3     int key;
4     Node next;
5 }

```

FIGURE 9.2

The `Node<T>` class: This internal class keeps track of the item, the item's key, and the next node in the list. Some algorithms require changes to this class.

**FIGURE 9.3**

A sequential Set<> implementation: adding and removing nodes. In part (a), a thread adding a node *b* uses two variables: *curr* is the current node, and *pred* is its predecessor. The thread moves down the list comparing the keys for *curr* and *b*. If a match is found, the item is already present, so it returns *false*. If *curr* reaches a node with a higher key, the item is not in the set, so it sets *b*'s next field to *curr*, and *pred*'s next field to *b*. In part (b), to delete *curr*, the thread sets *pred*'s next field to *curr*'s next field.

list. (Some of the algorithms we consider require changes to this class, such as adding new fields, changing the types of existing fields, or making some fields volatile.) For simplicity, we assume that each item's hash code is unique. (Relaxing this assumption is left as an exercise.) We associate an item with the same node and key throughout any given example, which allows us to abuse notation and use the same symbol to refer to a node, its key, and its item. That is, node *a* may have key *a* and item *a*, and so on.

In addition to nodes that hold items in the set, the list contains two *sentinel* nodes, *head* and *tail*, as the first and last nodes in the list. Sentinel nodes are never added, removed, or searched for, and their keys are the minimum and maximum integer values.¹ Ignoring synchronization for the moment, the top part of Fig. 9.3 schematically describes how an item is added to the set. A thread uses two local variables to traverse the list: *curr* is the current node and *pred* is its predecessor. To add an item to

¹ The algorithms presented here work for any ordered set of keys that has maximum and minimum values and is well founded, that is, there are only finitely many keys smaller than any given key. For simplicity, we assume here that keys are integers, and that no item's key is the maximum or minimum integer value.

the set, a thread sets *pred* to *head* and *curr* to its successor, and moves down the list, comparing *curr*'s key to the key of the item being added until it finds a node whose key is greater than or equal to the new item's key. If the keys match, the item is already present in the set, so the call returns *false*. Otherwise, *pred*'s key is less than that of the new item, and *curr*'s key is greater, so the item is not present in the list. The method creates a new node *b* to hold the item, sets *b*'s next field to *curr*, and then sets *pred*'s to *b*. Removing an item from the set works in a similar way.

9.3 Concurrent reasoning

Reasoning about concurrent data structures may seem impossibly difficult, but it is a skill that can be learned. Often, the key to understanding a concurrent data structure is to understand its *invariants*: properties that always hold. We can show that a property is invariant by showing that:

1. the property holds when the object is created, and
2. once the property holds, no thread can take a step that makes it *false*.

Most interesting invariants hold trivially when the list is created, so it makes sense to focus on how invariants, once established, are preserved.

Specifically, we can check that each invariant is preserved by each invocation of *insert()*, *remove()*, and *contains()* methods. This approach works only if we can assume that these methods are the *only* ones that modify nodes, a property sometimes called *freedom from interference*. In the list algorithms considered here, nodes are internal to the list implementation, so freedom from interference is guaranteed because users of the list have no opportunity to modify its internal nodes.

We require freedom from interference even for nodes that have been removed from the list, since some of our algorithms permit a thread to unlink a node while it is being traversed by others. Fortunately, we do not attempt to reuse list nodes that have been removed from the list, relying instead on a garbage collector to recycle that memory. The algorithms described here work in languages without garbage collection, but sometimes require nontrivial modifications that are beyond the scope of this chapter. We discuss issues that arise in the absence of garbage collection and how to deal with them in Chapter 19.

When reasoning about concurrent object implementations, it is important to understand the distinction between an object's *abstract value* (here, a set of items) and its *concrete representation* (here, a list of nodes).

Not every list of nodes is a meaningful representation for a set. An algorithm's *representation invariant* characterizes which representations make sense as abstract values. If *a* and *b* are nodes, we say that *a* *points to* *b* if *a*'s next field is a reference to *b*. We say that *b* is *reachable* if there is a sequence of nodes starting at *head* and ending at *b*, where each node in the sequence points to its successor.

The set algorithms in this chapter require the following invariants (some require more, as explained later):

1. The key of any node in the list is less than the key of its successor (if it has one). This implies that nodes in the list are sorted by key, and that keys are unique.
2. The key of any item added, removed, or searched for is greater than the key of head and less than the key of tail. (Hence, the sentinel nodes are neither added nor removed.)

Think of the representation invariant as a contract among the object's methods. Each method call preserves the invariant, and relies on the other methods to preserve the invariant. In this way, we can reason about each method in isolation, without having to consider all the possible ways they might interact.

Given a list satisfying the representation invariant, which set does it represent? The meaning of such a list is given by an *abstraction map* carrying lists that satisfy the representation invariant to sets. Here, the abstraction map is simple: An item is in the set if and only if it is (in a node) reachable from head.

What safety and liveness properties do we need? For safety, we want *linearizability*. As we saw in Chapter 3, to show that a concurrent data structure is a linearizable implementation of a sequential object, it suffices to identify a *linearization point*, an atomic step where the method call “takes effect”; we say it is *linearized* at this point. This step can be a read, a write, or a more complex atomic operation. Looking at any execution history of a list-based set, it must be the case that if the abstraction map is applied to the representation at the linearization points, the resulting sequence of states and method calls defines a valid sequential set execution. Here, `add(a)` adds *a* to the abstract set, `remove(a)` removes *a* from the abstract set, and `contains(a)` returns *true* or *false*, depending on whether *a* was already in the set.

Different list algorithms make different progress guarantees. Some use locks, and care is required to ensure they are deadlock- and starvation-free. Some *nonblocking* list algorithms do not use locks at all, while others restrict locking to certain methods. Here is a brief summary, from Chapter 3, of the nonblocking properties we use:²

- A method is *wait-free* if every call finishes in a finite number of steps.
- A method is *lock-free* if *some* call always finishes in a finite number of steps.

We are now ready to consider a variety of list-based set algorithms. We start with algorithms that use coarse-grained synchronization, and successively refine them to reduce the granularity of locking, culminating in a nonblocking algorithm. Formal proofs of correctness lie beyond the scope of this book. Instead, we focus on informal reasoning useful in everyday problem solving.

As mentioned, in each of these algorithms, methods traverse the list using two local variables: `curr` is the current node and `pred` is its predecessor. Because these variables are local, each thread has its own instances of them; we use `predA` and `currA` to denote the instances used by thread *A*.

² Chapter 3 introduces an even weaker nonblocking property called *obstruction-freedom*.

```

1 public class CoarseList<T> {
2     private Node head;
3     private Lock lock = new ReentrantLock();
4     public CoarseList() {
5         head = new Node(Integer.MIN_VALUE);
6         head.next = new Node(Integer.MAX_VALUE);
7     }
8     public boolean add(T item) {
9         Node pred, curr;
10        int key = item.hashCode();
11        lock.lock();
12        try {
13            pred = head;
14            curr = pred.next;
15            while (curr.key < key) {
16                pred = curr;
17                curr = curr.next;
18            }
19            if (key == curr.key) {
20                return false;
21            } else {
22                Node node = new Node(item);
23                node.next = curr;
24                pred.next = node;
25                return true;
26            }
27        } finally {
28            lock.unlock();
29        }
30    }

```

FIGURE 9.4

The CoarseList class: the add() method.

9.4 Coarse-grained synchronization

We start with a simple algorithm using coarse-grained synchronization. Figs. 9.4 and 9.5 show the add() and remove() methods for this coarse-grained algorithm. (The contains() method works in much the same way, and is left as an exercise.) The list itself has a single lock which every method call must acquire. The principal advantage of this algorithm, which should not be discounted, is that it is obviously correct. All methods act on the list only while holding the lock, so the execution is essentially sequential.

The linearization point for an add(*a*) or remove(*a*) call depends on whether the call was successful (i.e., whether *a* was already present). A successful add(*a*) call (*a* absent) is linearized at the point that it updates the next field of the predecessor of

```

31 public boolean remove(T item) {
32     Node pred, curr;
33     int key = item.hashCode();
34     lock.lock();
35     try {
36         pred = head;
37         curr = pred.next;
38         while (curr.key < key) {
39             pred = curr;
40             curr = curr.next;
41         }
42         if (key == curr.key) {
43             pred.next = curr.next;
44             return true;
45         } else {
46             return false;
47         }
48     } finally {
49         lock.unlock();
50     }
51 }

```

FIGURE 9.5

The `CoarseList` class: the `remove()` method. All methods acquire a single lock, which is released on exit by the `finally` block.

the node added (line 24). Similarly, a successful `remove(a)` call (i.e., if a is present before the call) is linearized when it updates the next field of the predecessor of the node removed (line 43). An unsuccessful `add(a)` or `remove(a)` method call, or any `contains(a)` call, can be linearized when the lock is acquired (or any time while the lock is held).³

The `CoarseList` class satisfies the same progress condition as its lock: If the Lock is starvation-free, so is our implementation. If contention is very low, this algorithm is an excellent way to implement a list. If, however, there is contention, then even if the lock itself performs well, threads will still be delayed waiting for one another.

9.5 Fine-grained synchronization

We can improve concurrency by locking individual nodes, rather than locking the list as a whole. Instead of placing a lock on the entire list, we add a Lock to each node,

³ We can linearize every method call at the instant it acquires the lock, but doing so requires a different abstraction map than the one described in Section 9.3.

along with `lock()` and `unlock()` methods. As a thread traverses the list, it locks each node when it first visits, and sometime later releases it. Such *fine-grained* locking permits concurrent threads to traverse the list together in a pipelined fashion.

Consider two nodes *a* and *b*, where *a* points to *b*. It is not safe to unlock *a* before locking *b* because another thread could remove *b* from the list in the interval between unlocking *a* and locking *b*. Instead, a thread acquires locks using a “hand-over-hand” protocol: it acquires the lock for a node (except the head node) while holding (i.e., before releasing) the lock for its predecessor. This locking protocol is sometimes called *lock coupling*. (Note that there is no obvious way to implement lock coupling using Java’s **synchronized** methods.)

Figs. 9.6 and 9.7 show the `FineList` algorithm’s `add()` and `remove()` methods. As in the coarse-grained list, `remove()` makes `currA` unreachable by setting `predA`’s `next`

```

1  public boolean add(T item) {
2      int key = item.hashCode();
3      head.lock();
4      Node pred = head;
5      try {
6          Node curr = pred.next;
7          curr.lock();
8          try {
9              while (curr.key < key) {
10                 pred.unlock();
11                 pred = curr;
12                 curr = curr.next;
13                 curr.lock();
14             }
15             if (curr.key == key) {
16                 return false;
17             }
18             Node node = new Node(item);
19             node.next = curr;
20             pred.next = node;
21             return true;
22         } finally {
23             curr.unlock();
24         }
25     } finally {
26         pred.unlock();
27     }
28 }

```

FIGURE 9.6

The `FineList` class: The `add()` method uses hand-over-hand locking to traverse the list. The **finally** blocks release locks before returning.


```

29 public boolean remove(T item) {
30     int key = item.hashCode();
31     head.lock();
32     Node pred = head;
33     try {
34         Node curr = pred.next;
35         curr.lock();
36         try {
37             while (curr.key < key) {
38                 pred.unlock();
39                 pred = curr;
40                 curr = curr.next;
41                 curr.lock();
42             }
43             if (curr.key == key) {
44                 pred.next = curr.next;
45                 return true;
46             }
47             return false;
48         } finally {
49             curr.unlock();
50         }
51     } finally {
52         pred.unlock();
53     }
54 }

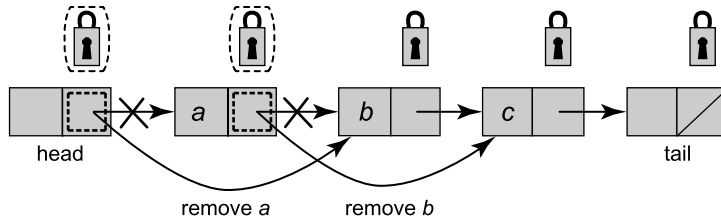
```

FIGURE 9.7

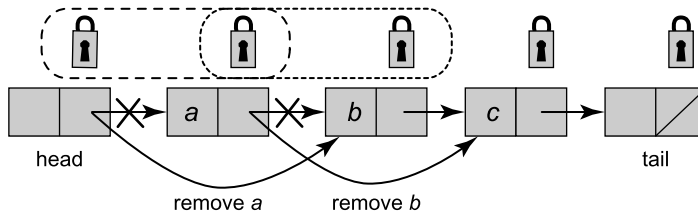
The `FineList` class: The `remove()` method locks both the node to be removed and its predecessor before removing that node.

field to curr_A 's successor. To be safe, `remove()` must lock both pred_A and curr_A . To see why, consider the scenario illustrated in Fig. 9.8. Thread *A* is about to remove node *a*, the first node in the list, while thread *B* is about to remove node *b*, where *a* points to *b*. Suppose *A* locks `head`, and *B* locks *a*. *A* then sets `head.next` to *b*, while *B* sets *a.next* to *c*. The net effect is the removal of *a*, but not *b*. The problem is that there is no overlap between the locks held by the two `remove()` calls. Fig. 9.9 illustrates how hand-over-hand locking avoids this problem.

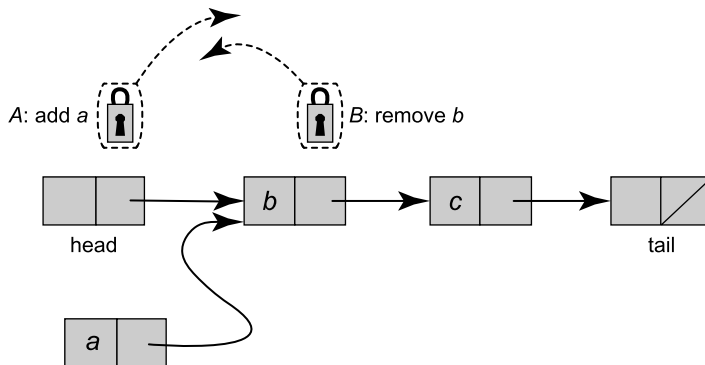
To guarantee progress, it is important that all methods acquire locks in the same order, starting at `head` and following next references toward `tail`. As Fig. 9.10 shows, a deadlock could occur if different method calls were to acquire locks in different orders. In this example, thread *A*, trying to add *a*, has locked *b* and is attempting to lock `head`, while *B*, trying to remove *b*, has locked `head` and is trying to lock *b*. Clearly, these method calls will never finish. Avoiding deadlocks is one of the principal challenges of programming with locks.

**FIGURE 9.8**

The `FineList` class: why `remove()` must acquire two locks. Thread *A* is about to remove *a*, the first node in the list, while thread *B* is about to remove *b*, where *a* points to *b*. Suppose *A* locks `head`, and *B* locks *a*. Thread *A* then sets `head.next` to *b*, while *B* sets *a*'s next field to *c*. The net effect is to remove *a*, but not *b*.

**FIGURE 9.9**

The `FineList` class: Hand-over-hand locking ensures that if concurrent `remove()` calls try to remove adjacent nodes, then they acquire conflicting locks. Thread *A* is about to remove node *a*, the first node in the list, while thread *B* is about to remove node *b*, where *a* points to *b*. Because *A* must lock both `head` and *a* and *B* must lock both *a* and *b*, they are guaranteed to conflict on *a*, forcing one call to wait for the other.

**FIGURE 9.10**

The `FineList` class: A deadlock could occur if, for example, `remove()` and `add()` calls acquired locks in opposite order. Then we could have the scenario depicted, in which thread *A* is about to insert *a* by locking first *b* and then `head`, and thread *B* is about to remove node *b* by locking first `head` and then *b*. Each thread holds the lock the other is waiting to acquire, so neither makes progress.

The representation invariant and abstraction map for `FineList` are the same as for `CoarseList`: Sentinels are never added or removed, nodes are sorted by key value without duplicates, and an item is in the set if and only if its node is reachable.

As in `CoarseList`, a successful `add(a)` or `remove(a)` call of `FineList` is linearized when it updates the `next` field of the predecessor of the node added or removed (line 20 or 44). An unsuccessful `add(a)` or `remove(a)` call, or any `contains(a)` call, can be linearized when it acquires the lock to a node whose key is greater than or equal to a (line 7 or 13 for `add(a)`; line 35 or 41 for `remove(a)`).

The `FineList` algorithm is starvation-free if all the node locks are starvation-free, but arguing this property is harder than in the coarse-grained case: Because all methods acquire locks in the same down-the-list order, deadlock is impossible. If thread A attempts to lock `head`, it eventually succeeds. From that point on, because there are no deadlocks, eventually all locks held by threads ahead of A in the list will be released, and A will succeed in locking `predA` and `currA`.

Although fine-grained locking reduces contention compared to coarse-grained locking, it imposes a potentially long sequence of lock acquisitions and releases. Moreover, threads accessing disjoint parts of the list may still block one another. For example, a thread removing the second item in the list blocks all concurrent threads searching for later nodes.

9.6 Optimistic synchronization

One way to reduce synchronization costs is to take a chance: Search without acquiring locks, lock the nodes found, and then confirm that the locked nodes are correct. If a synchronization conflict causes the wrong nodes to be locked, then release the locks and start over. When this kind of conflict is rare, this technique works well, which is why we call it *optimistic synchronization*.

Code for the optimistic `add(a)` method appears in Fig. 9.11. When thread A calls this method, it traverses the list without acquiring any locks (lines 6–8). In fact, it ignores the locks completely. It stops the traversal when `currA`'s key is greater than or equal to a . It then locks `predA` and `currA`, and calls `validate()` to check that `predA` is reachable and its `next` field still refers to `currA`. If validation succeeds, then thread A proceeds as before: If `currA`'s key is greater than a , thread A adds a new node with item a between `predA` and `currA`, and returns *true*. Otherwise it returns *false*. The `remove()` and `contains()` methods (Figs. 9.12 and 9.13) operate similarly, traversing the list without locking, and then locking the target nodes and validating they are still in the list. To be consistent with the Java memory model, the `next` fields in the nodes must be declared *volatile*.

The code of `validate()` appears in Fig. 9.14. We are reminded of a story:

A tourist takes a taxi in a foreign town. The taxi driver speeds through a red light. The tourist, frightened, asks, “What are you are doing?” The driver answers, “Do not worry, I am an expert.” He speeds through more red lights, and the tourist, on

```

1  public boolean add(T item) {
2      int key = item.hashCode();
3      while (true) {
4          Node pred = head;
5          Node curr = pred.next;
6          while (curr.key < key) {
7              pred = curr; curr = curr.next;
8          }
9          pred.lock();
10         try {
11             curr.lock();
12             try {
13                 if (validate(pred, curr)) {
14                     if (curr.key == key) {
15                         return false;
16                     } else {
17                         Node node = new Node(item);
18                         node.next = curr;
19                         pred.next = node;
20                         return true;
21                     }
22                 }
23             } finally {
24                 curr.unlock();
25             }
26         } finally {
27             pred.unlock();
28         }
29     }
30 }

```

FIGURE 9.11

The `OptimisticList` class: the `add()` method traverses the list ignoring locks, acquires locks, and validates before adding the new node.

the verge of hysteria, complains again, more urgently. The driver replies, “Relax, relax, you are in the hands of an expert.” Suddenly, the light turns green, the driver slams on the brakes, and the taxi skids to a halt. The tourist picks himself off the floor of the taxi and asks, “For crying out loud, why stop now that the light is finally green?” The driver answers, “Too dangerous, another expert could be crossing.”

Ignoring locks while traversing a dynamically changing lock-based data structure requires careful thought (there are other expert threads out there). We must be sure to use some form of *validation* and guarantee *freedom from interference*.

Validation is necessary because the trail of references leading to pred_A , or the reference from pred_A to curr_A , could have changed between when they were last

```

31 public boolean remove(T item) {
32     int key = item.hashCode();
33     while (true) {
34         Node pred = head;
35         Node curr = pred.next;
36         while (curr.key < key) {
37             pred = curr; curr = curr.next;
38         }
39         pred.lock();
40         try {
41             curr.lock();
42             try {
43                 if (validate(pred, curr)) {
44                     if (curr.key == key) {
45                         pred.next = curr.next;
46                         return true;
47                     } else {
48                         return false;
49                     }
50                 }
51             } finally {
52                 curr.unlock();
53             }
54         } finally {
55             pred.unlock();
56         }
57     }
58 }

```

FIGURE 9.12

The `OptimisticList` class: The `remove()` method traverses ignoring locks, acquires locks, and validates before removing the node.

read by A and when A acquired the locks. In particular, A could be traversing parts of the list that have already been removed. For example, as shown in Fig. 9.15, the node curr_A and all nodes between curr_A and a (including a) may be removed while A is still traversing curr_A . Thread A discovers that curr_A points to a ; without validation, it would “successfully” remove a , even though a is no longer in the list. A `validate()` call detects that a is no longer in the list, and the caller restarts the method.

Because we ignore the locks that protect concurrent modifications while traversing the list, a method call may traverse nodes that have been removed from the list. Nevertheless, freedom from interference implies that once a node has been unlinked from the list, the value of its `next` field does not change, so following a sequence of such links eventually leads back to the list. Freedom from interference, in turn, relies on garbage collection to ensure that no node is recycled while it is being traversed.

```

59 public boolean contains(T item) {
60     int key = item.hashCode();
61     while (true) {
62         Node pred = head;
63         Node curr = pred.next;
64         while (curr.key < key) {
65             pred = curr; curr = curr.next;
66         }
67         pred.lock();
68         try {
69             curr.lock();
70             try {
71                 if (validate(pred, curr)) {
72                     return (curr.key == key);
73                 }
74             } finally {
75                 curr.unlock();
76             }
77         } finally {
78             pred.unlock();
79         }
80     }
81 }

```

FIGURE 9.13

The OptimisticList class: The contains() method searches, ignoring locks, then acquires locks, and validates to determine if the node is in the list.

```

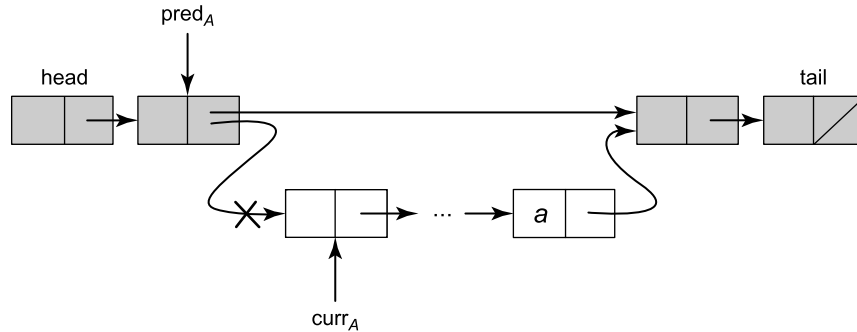
82 private boolean validate(Node pred, Node curr) {
83     Node node = head;
84     while (node.key <= pred.key) {
85         if (node == pred)
86             return pred.next == curr;
87         node = node.next;
88     }
89     return false;
90 }

```

FIGURE 9.14

The OptimisticList: Validation checks that pred points to curr and is reachable from head.

The linearization points of the optimistic algorithm are the same as those of the fine-grained algorithm, except that we must take into account the possibility of failed validation. In particular, we only linearize a method call when it acquires a lock if the subsequent validation succeeds.

**FIGURE 9.15**

The `OptimisticList` class: why validation is needed. Thread A is attempting to remove a node a . While traversing the list, curr_A and all nodes between curr_A and a (including a) might be removed (denoted by a lighter node color). In such a case, thread A would proceed to the point where curr_A points to a , and, without validation, would successfully remove a , even though it is no longer in the list. Validation is required to determine that a is no longer reachable from `head`.

The `OptimisticList` algorithm is not starvation-free, even if all node locks are individually starvation-free. A thread might be delayed indefinitely if new nodes are repeatedly added and removed (see Exercise 9.6). Nevertheless, we expect this algorithm to do well in practice, since starvation is rare. It works best if the cost of traversing the list twice without locking is significantly less than the cost of traversing the list once with locking.

Although optimistic synchronization can be applied to coarse-grained locking, it does not improve that algorithm because validation requires retraversing the list while holding the lock. However, we can eliminate the need to traverse the list during validation by maintaining a version number that is incremented whenever the list is modified. We explore such an approach in the exercises.

9.7 Lazy synchronization

One drawback of the `OptimisticList` algorithm (and the `CoarseList` and `FineList` algorithms) is that `contains()` acquires locks, which is unattractive since `contains()` calls are typically much more common than calls to other methods. We show how to refine this algorithm so that `contains()` is wait-free, and `add()` and `remove()` methods, while still blocking, traverse the list only once (in the absence of contention).

We add to each node a Boolean `marked` field indicating whether that node is in the set. Now, traversals do not need to lock the target node, and there is no need to validate that the node is reachable by retraversing the whole list. Instead, the algorithm maintains the invariant that every unmarked node is reachable. If a traversing thread does not find a node, or finds it marked, then that item is not in the set. As a result,

```

1  private boolean validate(Node pred, Node curr) {
2      return !pred.marked && !curr.marked && pred.next == curr;
3  }

```

FIGURE 9.16

The LazyList class: Validation checks that neither the pred nor the curr node has been logically deleted, and that pred points to curr.

contains() needs only one wait-free traversal. To add an element to the list, add() traverses the list, locks the target's predecessor, and inserts the node. The remove() method is lazy, taking two steps: It first marks the target node, *logically* removing it, and then redirects the next field of its predecessor, *physically* removing it.

In more detail, as in the OptimisticList algorithm, all methods traverse the list (possibly traversing logically and physically removed nodes) ignoring the locks. The add() and remove() methods lock the pred_A and curr_A nodes, and validates them, as before, except that the validate() method (Fig. 9.16) does not retrace the list to determine that the nodes are still in the list. Instead, because of the new invariant, it suffices to check that they are not marked. We must check that both pred_A and curr_A are not marked because pred_A is the node that is being modified. (See Fig. 9.19 for an illustration of why validation is necessary.)

Except for calling a different validate() method, the add() method of LazyList is exactly the same as that of OptimisticList. The remove() method (Fig. 9.17) has a small difference: it marks the node (line 18) before (physically) removing it from the list, maintaining the invariant that every unmarked node is reachable.

Logical removals require a small change to the abstraction map: An item is in the set if and only if it is referred to by an *unmarked* reachable node. Note that the path along which the node is reachable may contain marked nodes. (The diligent reader should check that any unmarked reachable node remains reachable, even if its predecessor is logically or physically deleted.) As in the OptimisticList algorithm, add() and remove() are not starvation-free, because list traversals may be arbitrarily delayed by ongoing modifications.

The contains() method (Fig. 9.18) traverses the list once ignoring locks and returns *true* if the node it was searching for is present and unmarked, and *false* otherwise. It is thus wait-free.⁴ A marked node's value is ignored. Each time the traversal moves to a new node, the new node has a larger key than the previous one, even if the node is logically deleted.

The linearization points for LazyList add() and unsuccessful remove() calls are the same as for the OptimisticList. A successful remove() call is linearized when the node is marked (i.e., when the marked bit is set on line 18), and a successful contains() call is linearized when an unmarked matching node is found.

⁴ The number of nodes a thread must traverse cannot increase without bound due to newly inserted nodes because the set of keys is well founded.


```

4  public boolean remove(T item) {
5      int key = item.hashCode();
6      while (true) {
7          Node pred = head;
8          Node curr = head.next;
9          while (curr.key < key) {
10             pred = curr; curr = curr.next;
11         }
12         pred.lock();
13         try {
14             curr.lock();
15             try {
16                 if (validate(pred, curr)) {
17                     if (curr.key == key) {
18                         curr.marked = true;
19                         pred.next = curr.next;
20                         return true;
21                     } else {
22                         return false;
23                     }
24                 }
25             } finally {
26                 curr.unlock();
27             }
28         } finally {
29             pred.unlock();
30         }
31     }
32 }

```

FIGURE 9.17

The LazyList class: The remove() method removes nodes in two steps, logical and physical.

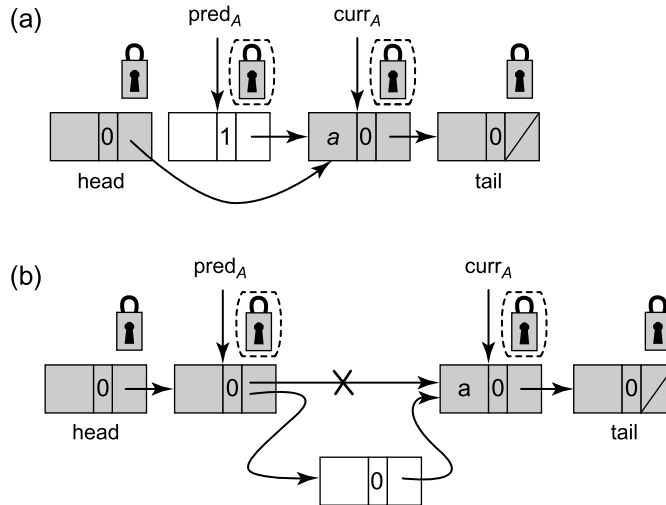
```

33 public boolean contains(T item) {
34     int key = item.hashCode();
35     Node curr = head;
36     while (curr.key < key)
37         curr = curr.next;
38     return curr.key == key && !curr.marked;
39 }

```

FIGURE 9.18

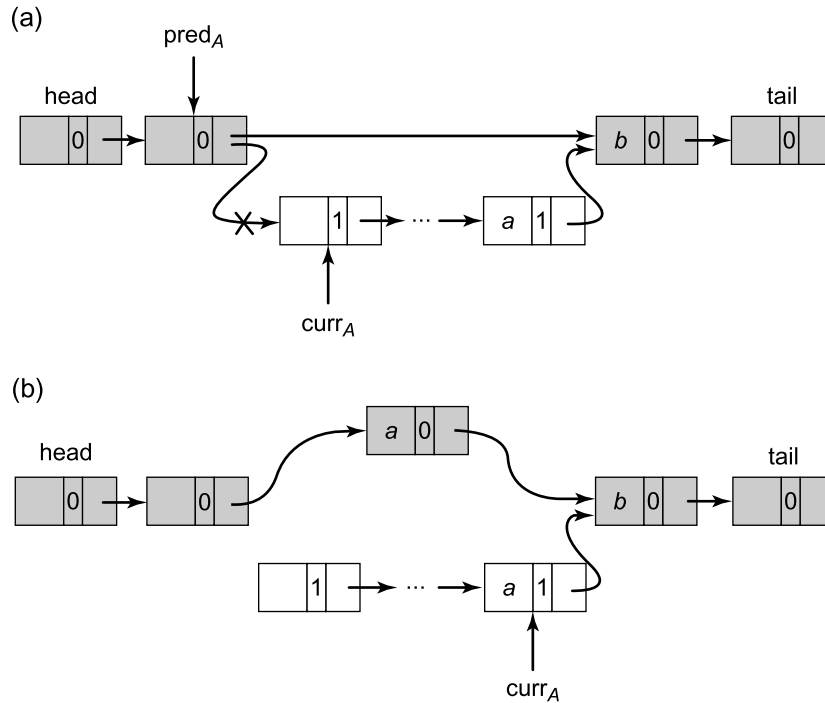
The LazyList class: the contains() method.

**FIGURE 9.19**

The LazyList class: why validation is needed. In part (a), thread A is attempting to remove node *a*. After it reaches the point where pred_A refers to curr_A , and before it acquires locks on these nodes, the node pred_A is logically and physically removed. After A acquires the locks, validation will detect the problem. In part (b), A is attempting to remove node *a*. After it reaches the point where pred_A refers to curr_A , and before it acquires locks on these nodes, a new node is added between pred_A and curr_A . After A acquires the locks, even though neither pred_A nor curr_A is marked, validation detects that pred_A is not the same as curr_A , and A's call to `remove()` will be restarted.

It is trickier to see how to linearize an unsuccessful `contains()` method call. Consider the scenario depicted in Fig. 9.20, in which thread A is executing `contains(a)`. In part (a), while A is traversing the list, another thread removes, both logically and physically, curr_A and all subsequent nodes up to and including *a*. Thread A will still follow the links until curr_A points to *a*, and detect that *a* is marked, and hence no longer in the abstract set. We might be tempted to linearize it at that point (i.e., when A executes line 38). However, as depicted in part (b), this is not always a valid linearization point: while A is traversing the removed section of the list, and before it reaches the removed node *a*, another thread may call `add(a)`, adding a new node with key *a* to the reachable part of the list. In this case, A's unsuccessful `contains(a)` method call cannot be linearized at the point it finds the marked node *a*, because this point occurs *after* the new node with key *a* has been inserted in the list. The unsuccessful method call must be linearized to a point before the new node is inserted.

We therefore linearize an unsuccessful `contains(a)` method call within its execution interval at the earlier of the following points: (1) the point where a marked node with key *a*, or a node with a key greater than *a*, is found, or (2) the point immediately before a new node with key *a* is added to the list. The second point is guaranteed to be within the execution interval because the insertion of the new node with the same

**FIGURE 9.20**

The LazyList class: linearizing an unsuccessful `contains()` call. Dark nodes are physically in the list and white nodes are physically removed. In part (a), while thread *A* is traversing the list, another thread disconnects the sublist referred to by `currA`. We can linearize *A*'s call at the point it sees that *a* is marked and is no longer in the abstract set. However, in part (b), while *A* is traversing the removed part of the list leading to the marked node *a*, another thread adds a new node with key *a*. It would be wrong to linearize *A*'s unsuccessful `contains(a)` call to when it found the marked node *a*, since this point occurs *after* the insertion of the new node with key *a* to the list.

key must have happened after the start of the `contains()` method, or the `contains()` method would have found that item. As can be seen, the linearization point of the unsuccessful `contains()` is determined by the ordering of events in the execution, and is not a predetermined point in the method's code, and indeed, may not even be at a point where the thread takes a step (e.g., it may be linearized when another thread takes a step).

One benefit of lazy synchronization is that we can separate unobtrusive logical steps, such as setting a flag, from disruptive physical changes to the structure, such as disconnecting a node. The example presented here is simple because we disconnect one node at a time. In general, however, delayed operations can be batched and performed lazily at a convenient time, reducing the overall disruptiveness of physical modifications to the structure.

A principal disadvantage of the LazyList algorithm is that `add()` and `remove()` calls are blocking: If one thread is delayed, then others may also be delayed.

9.8 Nonblocking synchronization

We have seen that we can avoid locking in `contains()` by marking nodes as logically removed before physically removing them from the list. We now show how to extend this idea to eliminate locks altogether, allowing all three methods, `add()`, `remove()`, and `contains()`, to be nonblocking. (The first two methods are lock-free and the last wait-free.)

A naïve approach would be to use `compareAndSet()` to change the next fields. For example, if thread *A* wants to remove `currA` from the list, it might call `compareAndSet()` to set `predA`'s next field to `currA`'s successor. Unfortunately, this idea does not work, as shown in Fig. 9.21. Part (a) shows a thread *A* attempting to remove a node *a* while thread *B* is adding a node *b*. Suppose *A* applies `compareAndSet()` to `head.next`, while *B* applies `compareAndSet()` to `a.next`. The net effect is that *a* is correctly deleted but *b* is not added to the list. In part (b), *A* attempts to remove *a*, the first node in the list, while *B* is about to remove *b*, where *a* points to *b*. Suppose

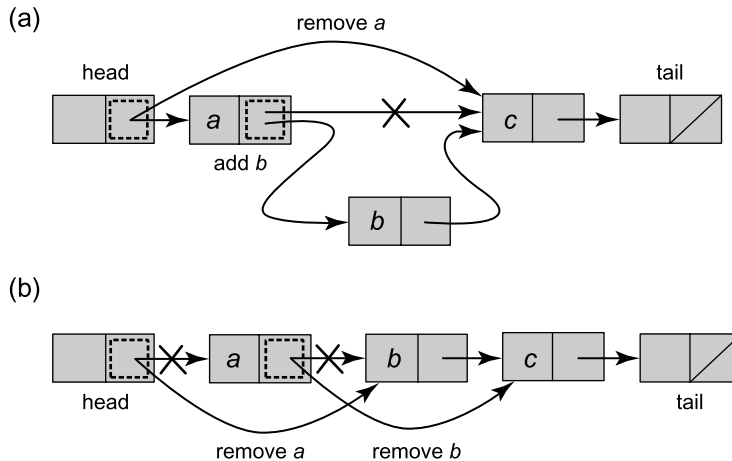


FIGURE 9.21

The `LockFreeList` class: why mark and reference fields must be modified atomically. In part (a), thread *A* is about to remove *a*, the first node in the list, while *B* is about to add *b*. Suppose *A* applies `compareAndSet()` to `head.next`, while *B* applies `compareAndSet()` to `a.next`. The net effect is that *a* is correctly deleted but *b* is not added to the list. In part (b), thread *A* is about to remove *a*, the first node in the list, while *B* is about to remove *b*, where *a* points to *b*. Suppose *A* applies `compareAndSet()` to `head.next`, while *B* applies `compareAndSet()` to `a.next`. The net effect is to remove *a*, but not *b*.

A applies `compareAndSet()` to `head.next`, while *B* applies `compareAndSet()` to `a.next`. The net effect is the removal of *a*, but not *b*.

We need a way to ensure that a node's fields cannot be updated after that node has been logically or physically removed from the list. Our approach is to treat the node's next and marked fields as a single atomic unit: any attempt to update the next field when the marked field is *true* will fail.

As described in detail in Pragma 9.8.1, an `AtomicMarkableReference<T>` object encapsulates both a reference to an object of type *T* and a Boolean mark. These fields can be atomically updated, either together or individually.

We make each node's next field an `AtomicMarkableReference<Node>`. Thread *A* logically removes `currA` by setting the mark bit in the node's next field, and shares the physical removal with other threads performing `add()` or `remove()`: As each thread traverses the list, it cleans up the list by physically removing any marked nodes it encounters. In other words, threads performing `add()` and `remove()` do not traverse

PRAGMA 9.8.1

An `AtomicMarkableReference<T>` object (defined by the `java.util.concurrent.atomic` package) encapsulates both a reference to an object of type *T* and a Boolean mark, also called a *mark bit*. These fields can be updated atomically, either together or individually. The `compareAndSet()` method tests the expected reference and mark values, and if both tests succeed, replaces them with updated reference and mark values. The `get()` method has an unusual interface: It returns the object's reference value and stores the mark value in a Boolean array argument. The `getReference()` and `isMarked()` methods return the reference and mark values, respectively. The interfaces of these methods are shown in Fig. 9.22.

In C or C++, one could provide this functionality efficiently by “stealing” a bit from a pointer, using bit-wise operators to extract the mark and the pointer from a single word. In Java, one cannot manipulate pointers directly, so this functionality must be provided by a library.

```

1 public boolean compareAndSet(T expectedReference,
2                             T newReference,
3                             boolean expectedMark,
4                             boolean newMark);
5 public T get(boolean[] marked);
6 public T getReference();
7 public boolean isMarked();

```

FIGURE 9.22

Some `AtomicMarkableReference<T>` methods: `compareAndSet()` tests and updates both the mark and reference fields; `get()` returns the encapsulated reference and stores the mark at position 0 in the argument array; `getReference()` and `isMarked()` return the reference and mark, respectively.

marked nodes; they remove them before continuing. The `contains()` method remains the same as in the `LazyList` algorithm, traversing all nodes whether they are marked or not, and testing if an item is in the list based on its key and mark.

It is worth pondering a design decision that differentiates the `LockFreeList` algorithm from the `LazyList` algorithm. Why do threads that add or remove nodes never traverse marked nodes, and instead physically remove all marked nodes they encounter? Suppose that thread A were to traverse marked nodes without physically removing them, and after logically removing curr_A , were to attempt to physically remove it as well. It could do so by calling `compareAndSet()` to try to redirect pred_A 's next field, simultaneously verifying that pred_A is not marked and that it refers to curr_A . The difficulty is that, because A is not holding locks on pred_A and curr_A , other threads could insert new nodes or remove pred_A before the `compareAndSet()` call.

Consider a scenario in which another thread marks pred_A . As illustrated in Fig. 9.21, we cannot safely redirect the next field of a marked node, so A would have to restart the physical removal by retraversing the list. This time, however, A would have to physically remove pred_A before it could remove curr_A . Even worse, if there is a sequence of logically removed nodes leading to pred_A , A must remove them all, one after the other, before it can remove curr_A itself.

This example illustrates why `add()` and `remove()` calls do not traverse marked nodes: When they arrive at the node to be modified, they may be forced to retrace the list to remove previous marked nodes. Instead, we choose to have both `add()` and `remove()` physically remove any marked nodes on the path to their target node. The `contains()` method, by contrast, performs no modification, and therefore need not participate in the cleanup of logically removed nodes, allowing it, as in the `LazyList`, to traverse both marked and unmarked nodes.

In presenting the `LockFreeList` algorithm, we factor out functionality common to the `add()` and `remove()` methods by creating a nested `Window` class to help with traversal. As shown in Fig. 9.23, a `Window` object is a structure with `pred` and `curr` fields. The `find()` method takes a head node and a key a , and traverses the list, seeking to set `pred` to the node with the largest key less than a , and `curr` to the node with the least key greater than or equal to a . As thread A traverses the list, each time it advances curr_A , it checks whether that node is marked (line 16). If so, it calls `compareAndSet()` to attempt to physically remove the node by setting pred_A 's next field to curr_A 's next field. This call tests both the field's reference and Boolean mark values, and fails if either value has changed. A concurrent thread could change the mark value by logically removing pred_A , or it could change the reference value by physically removing curr_A . If the call fails, A restarts the traversal from the head of the list; otherwise the traversal continues.

The `LockFreeList` class uses the same abstraction map as the `LazyList` class: An item is in the set if and only if it is referred to by an *unmarked* reachable node. The `compareAndSet()` call at line 17 of the `find()` method is an example of a *benevolent side effect*: It changes the concrete list without changing the abstract set, because removing a marked node does not change the value of the abstraction map.

```

1  class Window {
2      public Node pred, curr;
3      Window(Node myPred, Node myCurr) {
4          pred = myPred; curr = myCurr;
5      }
6  }
7  Window find(Node head, int key) {
8      Node pred = null, curr = null, succ = null;
9      boolean[] marked = {false};
10     boolean snip;
11     retry: while (true) {
12         pred = head;
13         curr = pred.next.getReference();
14         while (true) {
15             succ = curr.next.get(marked);
16             while (marked[0]) {
17                 snip = pred.next.compareAndSet(curr, succ, false, false);
18                 if (!snip) continue retry;
19                 curr = succ;
20                 succ = curr.next.get(marked);
21             }
22             if (curr.key >= key)
23                 return new Window(pred, curr);
24             pred = curr;
25             curr = succ;
26         }
27     }
28 }

```

FIGURE 9.23

The LockFreeList class: nested Window class and find() method: find() returns a Window object with nodes on either side of the key; it removes marked nodes that it encounters.

Fig. 9.24 shows the LockFreeList class's add() method. Suppose thread A calls add(*a*). A uses find() to locate *pred_A* and *curr_A*. If *curr_A*'s key is equal to *a*'s, the call returns *false*. Otherwise, add() initializes a new node *a* to hold *a*, and makes *a* point to *curr_A*. It then calls compareAndSet() (line 39) to make *pred_A* point to *a*. Because the compareAndSet() tests both the mark and the reference, it succeeds only if *pred_A* is unmarked and points to *curr_A*. If the compareAndSet() is successful, the method returns *true*; otherwise it starts over from head.

Fig. 9.25 shows the LockFreeList algorithm's remove() method. When A calls remove() to remove item *a*, it uses find() to locate *pred_A* and *curr_A*. If *curr_A*'s key fails to match *a*'s, the call returns *false*. Otherwise, remove() uses a compareAndSet() to attempt to mark *curr_A* as logically removed (line 55). This call succeeds only if no other thread has set the mark first. If it succeeds, the call returns *true*. A single attempt is made to physically remove the node, but there is no need to try again

```

29 public boolean add(T item) {
30     int key = item.hashCode();
31     while (true) {
32         Window window = find(head, key);
33         Node pred = window.pred, curr = window.curr;
34         if (curr.key == key) {
35             return false;
36         } else {
37             Node node = new Node(item);
38             node.next = new AtomicMarkableReference(curr, false);
39             if (pred.next.compareAndSet(curr, node, false, false)) {
40                 return true;
41             }
42         }
43     }
44 }

```

FIGURE 9.24

The `LockFreeList` class: The `add()` method calls `find()` to locate `pred` and `curr`. It adds a new node only if `pred` is unmarked and refers to `curr`.

```

45 public boolean remove(T item) {
46     int key = item.hashCode();
47     boolean snip;
48     while (true) {
49         Window window = find(head, key);
50         Node pred = window.pred, curr = window.curr;
51         if (curr.key != key) {
52             return false;
53         } else {
54             Node succ = curr.next.getReference();
55             snip = curr.next.compareAndSet(succ, succ, false, true);
56             if (!snip)
57                 continue;
58             pred.next.compareAndSet(curr, succ, false, false);
59             return true;
60         }
61     }
62 }

```

FIGURE 9.25

The `LockFreeList` class: The `remove()` method calls `find()` to locate `pred` and `curr`, and atomically marks the node for removal.


```

63 public boolean contains(T item) {
64     int key = item.hashCode();
65     Node curr = head;
66     while (curr.key < key) {
67         curr = curr.next.getReference();
68     }
69     return (curr.key == key && !curr.next.isMarked())
70 }

```

FIGURE 9.26

The `LockFreeList` class: The wait-free `contains()` method is the same as in the `LazyList` class, except that it calls `curr.next.getReference()` to get the successor of `curr` and `curr.next.isMarked()` to test whether `curr` is marked.

because the node will be removed by the next thread to traverse that region of the list. If the `compareAndSet()` call fails, `remove()` starts over.

The `contains()` method of the `LockFreeList` algorithm, shown in Fig. 9.26, is the same as that of the `LazyList` algorithm, except that it uses `curr.next.getReference()` and `curr.next.isMarked()` to get the successor and mark bit of `curr`.

9.9 Discussion

We have seen a progression of list-based lock implementations in which the granularity and frequency of locking was gradually reduced, eventually reaching a fully nonblocking list. The final transition from the `LazyList` to the `LockFreeList` exposes some of the design decisions that concurrent programmers face. As we will see, approaches such as optimistic and lazy synchronization will appear time and again when designing more complex data structures.

On the one hand, the `LockFreeList` algorithm guarantees progress in the face of arbitrary delays. However, there is a price for this strong progress guarantee:

- The need to support atomic modification of a reference and a Boolean mark has an added performance cost.⁵
- As `add()` and `remove()` traverse the list, they engage in concurrent cleanup of removed nodes, introducing the possibility of contention among threads, sometimes forcing threads to restart traversals, even if there was no change near the node each was trying to modify.

On the other hand, the lazy lock-based list does not guarantee progress in the face of arbitrary delays: Its `add()` and `remove()` methods are blocking. However, unlike the

⁵ In the `java.util.concurrent` package, this cost is somewhat reduced by using a reference to an intermediate dummy node to signify that the marked bit is set.

lock-free algorithm, it does not require each node to include an atomically markable reference. It also does not require traversals to clean up logically removed nodes; they progress down the list, ignoring marked nodes.

Which approach is preferable depends on the application. In the end, the balance of factors such as the potential for arbitrary thread delays, the relative frequency of calls to the `add()` and `remove()` methods, the overhead of implementing an atomically markable reference, and so on, determines the choice of whether to lock, and if so, at what granularity.

9.10 Chapter notes

Lock coupling was invented by Rudolf Bayer and Mario Schkolnick [17]. The first designs of lock-free linked list algorithms are credited to John Valois [164]. The lock-free list implementation shown here is a variation on the lists of Maged Michael [126], who based his work on earlier linked list algorithms by Tim Harris [58]. This algorithm, referred to by many as the Harris–Michael algorithm, is the one used in the `java.util.concurrent` package. The `OptimisticList` algorithm was invented for this chapter, and the lazy algorithm is credited to Steve Heller, Maurice Herlihy, Victor Luchangco, Mark Moir, Bill Scherer, and Nir Shavit [60].

9.11 Exercises

Exercise 9.1. Describe how to modify each of the linked list algorithms if object hash codes are not guaranteed to be unique.

Exercise 9.2. Suppose every method call of `CoarseList` is linearized at the instant the lock is acquired. Explain why we cannot use the abstraction map described in Section 9.3. Give an alternative abstraction map that works for these linearization points.

Exercise 9.3. Explain why the fine-grained locking algorithm is does not deadlock.

Exercise 9.4. Explain why the fine-grained list’s `add()` method is linearizable.

Exercise 9.5. Explain why the optimistic and lazy locking algorithms are not subject to deadlock.

Exercise 9.6. Show an execution of the optimistic algorithm in which a thread is forever attempting to delete a node.

Hint: Since we assume that all the individual node locks are starvation-free, the live-lock is not on any individual lock, and a bad execution must repeatedly add and remove nodes from the list.

Exercise 9.7. Provide the code for the `contains()` method missing from the fine-grained algorithm. Explain why your implementation is correct.

Exercise 9.8. Is the optimistic list implementation still correct if we switch the order in which `add()` locks the `pred` and `curr` nodes?

Exercise 9.9. Show that in the optimistic list algorithm, if `predA` is not *null*, then `tail` is reachable from `predA`, even if `predA` itself is not reachable.

Exercise 9.10. Show that in the optimistic algorithm, the `add()` method needs to lock only `pred`.

Exercise 9.11. Design a coarse-grained optimistic locking linked list-based set algorithm that does not traverse the list while holding the lock by augmenting the list with a version number.

Exercise 9.12. Design a fine-grained optimistic locking algorithm that uses a version number to avoid traversing the list while holding any lock if the list does not change during the first traversal of the list. What are the advantages and disadvantages of this list compared with the coarse-grained list from the previous exercise?

Exercise 9.13. For each of the following modifications of the sorted linked list algorithms, explain why the respective algorithm is still linearizable, or give a counterexample showing it is not.

- a. In the optimistic algorithm, the `contains()` method locks two nodes before deciding whether a key is present. Suppose, instead, it locks no nodes, returning *true* if it observes the value, and *false* otherwise.
- b. In the lazy algorithm, the `contains()` method executes without inspecting the locks, but it inspects the mark bit; it returns *false* if a node is marked for removal. Suppose, instead, the `contains()` does not inspect the mark bit of the nodes, and returns *true* even for nodes that may be marked.

Exercise 9.14. Would the lazy algorithm still work if we marked a node as removed simply by setting its `next` field to *null*? Why or why not? What about the lock-free algorithm?

Exercise 9.15. In the lazy algorithm, can `predA` ever be unreachable? Justify your answer.

Exercise 9.16. Your new employee claims that the lazy list's validation method (Fig. 9.16) can be simplified by dropping the check that `pred.next` is equal to `curr`. After all, the code always sets `pred` to the old value of `curr`, and before `pred.next` can be changed, the new value of `curr` must be marked, causing the validation to fail. Explain the error in this reasoning.

Exercise 9.17. Can you modify the lazy algorithm's `remove()` so it locks only one node?

Exercise 9.18. In the lock-free algorithm, argue the benefits and drawbacks of having the `contains()` method help in the cleanup of logically removed nodes.

Exercise 9.19. In the lock-free algorithm, if an `add()` method call fails because `pred` does not point to `curr`, but `pred` is not marked, do we need to traverse the list again from `head` in order to attempt to complete the call?

Exercise 9.20. Would the `contains()` method of the lazy and lock-free algorithms still be correct if logically removed entries were not guaranteed to be sorted?

Exercise 9.21. The `add()` method of the lock-free algorithm never finds a marked node with the same key. Can one modify the algorithm so that it will simply insert its new added object into the existing marked node with the same key if such a node exists in the list, thus saving the need to insert a new node?

Exercise 9.22. Explain why the following cannot happen in the `LockFreeList` algorithm: A node with item x is logically but not yet physically removed by some thread, then the same item x is added into the list by another thread, and finally a `contains()` call by a third thread traverses the list, finding the logically removed node, and returning *false*, even though the linearization order of the `remove()` and `add()` implies that x is in the set.

Exercise 9.23. Consider the following two modifications for the sorted linked list algorithms:

- a. In the optimistic algorithm, the `contains()` method locks two nodes before deciding whether a key is present. Suppose, instead, it locks no nodes, returning *true* if it observes the value, and *false* otherwise.
- b. In the lazy algorithm, the `contains()` method executes without inspecting the locks, but it inspects the mark bit; it returns *false* if a node is marked for removal. Suppose, instead, the `contains()` does not inspect the mark bit of the nodes, and returns *true* even for nodes that may be marked.

For both of the modifications, explain why the respective algorithm is still linearizable, or give a counterexample showing it is not.

Exercise 9.24. In the lock-free algorithm, we attempt to logically remove the node `curr` by calling `curr.next.compareAndSet(succ, succ, false, true)` (line 55 of Fig. 9.25). For each of the following implementations in which this call is replaced with a different method call, either explain why it is correct or describe an execution in which it fails.

- a. We instead call `curr.next.compareAndSetMark(false, true)`, where `compareAndSetMark()` is a fictional method that atomically performs a normal compare-and-swap operation on just the mark bit.
- b. We instead call `curr.next.attemptMark(succ, true)`, where `attemptMark()` is a real method of the `AtomicMarkableReference<T>` class that atomically changes the mark bit to the specified value if the reference has the expected value, but is allowed to spuriously fail (if there are concurrent modifications).